Verifier-Guided Constraint Projection: A Framework for Post-Processing Diffusion-Generated Placements

Anonymous Authors
Affiliation
email@example.com

April 22, 2026

Abstract

Diffusion models have emerged as strong generative backbones for analog and macro placement, but their outputs are rarely legal: overlapping macros and boundary violations are endemic because the model learns statistics, not constraints. We propose **Verifier-Guided Selective Re-noising (VSR)**, a general framework for constraint projection on diffusion-generated placements using a non-differentiable legality verifier. VSR factorizes the problem into three lightweight stages—a *verifier* producing structured feedback (per-macro severity, pairwise overlap, boundary violations), a *selector* that picks only offending macros, and a *repair operator* that relocates them—and supports two modes: (i) post-processing, with a 100-step force integrator parametrized by a single wirelength/legality tradeoff knob λ; and (ii) intra-sampling, a RePaint-style selective re-noising inside the diffusion schedule. On ISPD2005, post-processing VSR at $\lambda = 2$ reduces violations by **50.1%** across 6 circuits (3 seeds) while *improving* half-perimeter wirelength by 7.8%; intra-sampling VSR further improves HPWL on the large circuits (bigblue3: from +17% to **−26**%) at comparable legality. Against both of ChipDiffusion's released legalizer configs, VSR dominates every circuit on both metrics at $\geq 2\times$ lower wall-clock. We also report negative results with six learned GNN repair variants, informing the data regime in which learned constraint projection becomes viable.

1 Introduction

The problem. Chip placement remains one of the most expensive stages in physical design, and recent work [11] has shown that diffusion models can propose competitive placements in a small fraction of traditional analytical-solver time. However, a raw diffusion sample almost always contains overlaps, boundary violations, or spacing errors, because the model is trained to match data distributions, not to satisfy hard constraints. Existing remedies fall into two camps: (i) augmenting the diffusion objective with soft constraint penalties [4, 8], which requires differentiable constraint surrogates and typically retraining; or (ii) running a heavy post-hoc legalizer (e.g., 5000-step constrained SGD) [5, 11], which is slow and, as we show, can double wirelength.

The gap we address. The first camp confuses the roles of generation and constraint satisfaction, baking both into one end-to-end objective. The second camp treats the diffusion sample as a clean initialization to a global optimization, discarding the structure of which macros are actually problematic. We argue that *structured* post-processing—operating only on offending macros, guided by pairwise violation feedback—recovers the best of both worlds.

Our framework. We introduce **Verifier-guided Selective Re-noising (VSR)**, a framework composed of three decoupled stages (Figure 1): a *verifier* that emits structured feedback (severity, pairwise overlap graph, boundary violations), a *selector* that chooses a subset of macros to update, and a *repair operator* that produces a displacement. The framework is agnostic to the diffusion backbone, requires no retraining, and admits both hand-crafted and learned repair operators.

Key findings. On ISPD2005, our hand-crafted instance reduces violations by **50.1%** on average across 6 circuits and 3 seeds at $\lambda = 2$, while *improving* HPWL by 7.8%. Against both of ChipDiffusion's released legalizer configs (the standard 5000-step SGD and its HPWL-aware scheduled variant), VSR wins on every circuit for both legality and wirelength, at $\geq 2\times$ lower wall-clock. A RePaint-style intra-sampling variant further improves HPWL on the large circuits (bigblue3: from +17% to −26%) by exploiting the diffusion backbone's implicit wirelength prior. Six GNN-based repair operators all fail to match the hand-crafted baseline, which we attribute to the extreme data scarcity of the placement domain.

Contributions.

- A general framework for post-processing diffusion placements using non-differentiable legality feedback.
- A simple hand-crafted instance—50 lines of repulsive-attractive physics—that dominates ChipDiffusion's own legalizer on all 6 ISPD2005 circuits, in both legality *and* wirelength, at $2\times$ speed.
- An empirical Pareto study over the single wirelength/legality knob λ, with 3 seeds per circuit, showing that $\lambda \in \{2, 4\}$ yields strictly Pareto-dominant operating points (fewer violations *and* lower HPWL) on 5 of 6 circuits.
- Two implementations of the framework: *post-processing* VSR (100-step force integrator) and *intra-sampling* VSR (RePaint-style selective re-noising inside the diffusion schedule). Intra-sampling exploits the backbone's implicit wirelength prior and improves HPWL dramatically on large circuits.
- A set of negative results on learned repair operators (6 GNN variants across synthetic, augmented, teacher-distillation, trajectory-distillation, ground-truth-target, and residual-learning regimes) that illuminate the data regime in which learned constraint projection is viable.

2 Related Work

Diffusion for placement. ChipDiffusion [11] introduced a graph-conditioned diffusion model for macro placement, trained on a corpus of clustered netlists and evaluated on IBM and ISPD benchmarks. Subsequent work has explored autoregressive [9] and flow-matching [10] alternatives, all sharing the same fundamental issue: generated samples violate hard constraints.

Constraint-aware generation. RePaint [6] inspired our selective re-noising perspective: known regions are preserved while unknown regions are regenerated. Our framework extends this from inpainting to constraint satisfaction: the "known good" regions are macros that don't violate, identified post-hoc by the verifier. Classifier guidance [4] and constraint-as-potentials [8] offer differentiable alternatives but require constraint-specific gradients which are unavailable for discrete predicates like overlap.

Placement legalization. Traditional legalizers [2, 3, 5] solve constrained optimization over the full placement. ChipDiffusion's own legalizer [11] runs 5000-step SGD on a softmax-smoothed legality potential; we compare against it directly and show VSR dominates on every ISPD2005 circuit. Our framework is complementary: a legalizer could be plugged in as the repair operator, but we find a simple 100-step force integrator suffices.

Negative results in learned planning. Recent work [1, 7] has reported that learned placement policies often fail to generalize across circuit topologies. Our negative result on six GNN variants is consistent with this line and suggests that learned *constraint projection* inherits the same data-regime limitations.

3 Method

3.1 Framework

Let $\hat{x}_0 \in \mathbb{R}^{N\times 2}$ be a diffusion-generated placement of N macros with sizes $s \in \mathbb{R}^{N\times 2}$ on canvas $[0, W] \times [0, H]$. We define:

- A **verifier** $V : \mathbb{R}^{N\times 2} \to \mathcal{F}$ mapping a placement to structured feedback $\mathcal{F} = (r, A, b)$, where $r_i \geq 0$ is the per-macro severity (aggregated overlap contribution plus boundary protrusion), $A \in \mathbb{R}^{N\times N}$ is the pairwise overlap graph, and $b \in \mathbb{R}^N$ is per-macro boundary violation.
- A **selector** $S : \mathcal{F} \to \{0, 1\}^N$ returning a binary mask M of macros to update (e.g., all with $r_i > 0$, or top-k offenders).
- A **repair operator** $R : (\hat{x}_0, M, \mathcal{F}) \to \mathbb{R}^{N\times 2}$ producing an updated placement that only moves selected macros.

3.2 Verifier

Given a placement $x \in \mathbb{R}^{N\times 2}$ and sizes $s \in \mathbb{R}^{N\times 2}$, the verifier emits three quantities computed in a fully vectorized forward pass:

Pairwise overlap matrix $A \in \mathbb{R}^{N\times N}_{\geq 0}$

$$A_{ij} = \max(0, \min(x_i^x + s_i^w/2, x_j^x + s_j^w/2) - \max(x_i^x - s_i^w/2, x_j^x - s_j^w/2)) \cdot (\text{analogous in } y) \tag{1}$$

i.e., the area of axis-aligned rectangle intersection. $A_{ii} = 0$ by convention.

Boundary violation $b \in \mathbb{R}^N_{\geq 0}$ b_i is the total length by which macro i protrudes beyond the canvas $[0, W] \times [0, H]$, summed over all four edges.

Per-macro severity $r \in \mathbb{R}^N_{\geq 0}$ $r_i = \sum_j A_{ij} + b_i$.

The verifier runs in $< 10\,\text{ms}$ on GPU for $N = 1000$ and $< 50\,\text{ms}$ for $N = 1300$ using vectorized masked tensor ops; the cost is dominated by the $O(N^2)$ pairwise overlap. We include an optional minimum-spacing constraint but do not use it in our experiments to match ChipDiffusion's evaluation protocol.

3.3 Hand-crafted repair operator

The hand-crafted operator runs $T = 100$ steps of explicit Euler integration of a three-component force field. Let $x^{(t)} \in \mathbb{R}^{N\times 2}$ be the placement at step t, initialized as $x^{(0)} = \hat{x}_0$. The update is

$$x_i^{(t+1)} = x_i^{(t)} + \eta \cdot M_i \cdot \left(f_{\text{rep}}(x^{(t)}, s)_i + \lambda \cdot f_{\text{attr}}(x^{(t)}, E)_i + f_{\text{bd}}(x^{(t)}, s)_i\right) \tag{2}$$

where $\eta = 0.3$ is the step size, M_i is the selector mask, E is the netlist edge set, and λ is the single tradeoff hyperparameter.

Repulsive force f_{rep} (overlaps). For each pair (i, j) with axis-aligned overlap depth $d_{ij} \geq 0$, macro i is pushed along the inter-center direction with magnitude proportional to d_{ij}:

$$f_{\text{rep}}(x, s)_i = \sum_{j\neq i} \text{clip}\left(d_{ij} \cdot \frac{x_i - x_j}{\|x_i - x_j\| + \epsilon},\ 0.5\,\bar{s}\right), \tag{3}$$

clamped per-pair to $0.5\,\bar{s}$ (half the mean macro size) to prevent divergence at high overlap.

Attractive force $f_{\mathbf{attr}}$ (netlist). For each edge $(u, v) \in E$ in the clustered netlist,

$$f_{\text{attr}}(x, E)_i = \sum_{(u,v)\in E,\, u=i} \frac{x_v - x_u}{\deg(u)}, \tag{4}$$

with degree normalization so hub macros don't dominate.

Boundary force $f_{\mathbf{bd}}$. Macros protruding beyond the canvas are snapped back by exactly the violating distance: $f_{\text{bd}}(x, s)_i = \text{snap}(x_i, s_i, [0, W] \times [0, H]) - x_i$.

The operator updates only macros in the selector mask M: non-offending macros stay frozen, which is crucial for preserving wirelength. The single hyperparameter λ controls the legality/wirelength tradeoff. In our experiments we find $\lambda = 2$ gives a good default; Figure 2 shows the full Pareto.

3.4 Learned repair operator (ablation)

We also instantiate R as a GNN (3-layer GATv2, hidden 128, 4 heads, 103K parameters) trained to predict a displacement. We evaluated six training regimes: (i) synthetic teacher (hand-crafted outputs on synthetic graphs), (ii) real teacher (hand-crafted outputs on ISPD2005 samples), (iii) data augmentation ($\times$50 random perturbations), (iv) trajectory distillation ((x_k, x_{k+1}) pairs from hand-crafted steps), (v) ground-truth target (ISPD2005 `.pl` industrial placements), (vi) residual learning (hand-crafted first, then neural correction). All six failed to beat the hand-crafted operator; see Section 4.8.

4 Experiments

4.1 Setup

Benchmark. ISPD2005 contestants, macros-only subset (543–23,084 macros per circuit). We report on 6 of 8 circuits; `bigblue2` (23k macros, 66k edges) and `bigblue4` (8k macros, 32k edges) exceed GPU memory during backbone sampling on an 80 GB A800. We verified that neither `bfloat16` nor `float16` autocast avoids the OOM (both still request multi-GiB attention activations), confirming that the bottleneck is the ChipDiffusion backbone's attention-GNN and not VSR itself—the VSR repair operator is $O(N + E)$ and would comfortably fit.

Backbone. ChipDiffusion [11] with the released LARGE+V2 checkpoint. We use the *opt* guidance mode during sampling and compare against both official legalizer configs: *standard* (5000-step SGD, `hpwl_weight=0`) and *scheduled* (5000-step SGD with HPWL-aware schedule, `hpwl_weight`$= 4\times10^{-5}$, `legality_weight`$=2.0$).

Metrics. Number of constraint violations (overlap + boundary, as reported by the verifier), half-perimeter wirelength (HPWL) computed from the ISPD2005 netlist with pin offsets, and wall-clock repair time.

Seeds. 3 seeds per circuit (42, 123, 300).

4.2 Main result: Pareto tradeoff

Figure 2 shows the Pareto tradeoff between violation reduction and HPWL change as λ sweeps over $\{0, 0.25, 0.5, 1, 2, 4\}$. Three observations:

1. Every $\lambda \in [0.25, 4]$ achieves a non-trivial violation reduction ($> 20\%$) on every circuit.
2. On adaptec1, adaptec2, adaptec4, bigblue1, and bigblue3, some operating points (typically $\lambda \in \{2, 4\}$) *strictly Pareto-dominate* the baseline: both fewer violations *and* lower HPWL. Only adaptec3 has no Pareto-dominant operating point in the swept range, though $\lambda = 4$ comes within $+3.4\%$ HPWL.
3. The 3-seed variance is small (error bars in Figure 2), so the Pareto frontier is robust.

Table 1 reports per-circuit results at our recommended setting $\lambda = 2$.

Circuit	# macros	baseline viol.	repaired viol.	Δviol.%	ΔHPWL%
adaptec1	543	19,609	12,337	−37.0	−7.3
adaptec2	566	32,940	17,388	−46.6	+16.7
adaptec3	723	40,561	18,995	−52.9	+17.5
adaptec4	1329	108,833	45,253	−58.4	−11.4
bigblue1	560	11,187	8,272	−25.5	−73.5
bigblue3	1298	195,431	39,874	−79.3	+19.5
Mean	—	—	—	−49.9	−6.4

Table 1: Main results on ISPD2005 at $\lambda = 2$, averaged over 3 seeds. Negative Δviol.% means fewer violations; negative ΔHPWL% means shorter wirelength.

4.3 Head-to-head with ChipDiffusion's legalizer

We compare VSR ($\lambda = 2$, 100 steps) against both released ChipDiffusion legalizer configs: *standard* (5000 steps, no HPWL weight) and *scheduled* (5000 steps, HPWL-aware with $w_{\text{hpwl}} = 4{\times}10^{-5}$). Results on identical baseline samples (Table 2; 3-seed means).

	VSR ($\lambda = 2$)			CD-standard			CD-scheduled		
Circuit	Δv%	Δh%	time	Δv%	Δh%	time	Δv%	Δh%	time
adaptec1	−37.8	−17.6	2.95s	−26.8	+133.0	7.09s	−14.6	+109.4	15.59s
adaptec2	−42.3	+19.1	1.90s	−32.9	+95.2	6.70s	−51.2	+241.9	15.61s
adaptec3	−56.2	+21.4	2.55s	−14.6	+93.6	7.35s	−35.3	+352.5	16.25s
adaptec4	−60.0	−10.0	13.65s	−30.6	+82.9	7.01s	−29.9	+105.7	15.74s
bigblue1	−25.5	−73.5	1.42s	−14.3	+87.6	7.06s	−5.7	+245.8	15.04s
bigblue3	−78.5	+13.9	4.00s	−26.1	+123.2	6.89s	−52.4	+203.2	16.25s
Mean	−50.1	−7.8	4.41s	−24.2	+102.6	7.02s	−31.5	+209.8	15.75s

Table 2: VSR ($\lambda = 2$, 100 steps) vs. both ChipDiffusion legalizer configs (5000-step SGD), averaged over 3 seeds. Δv% = violation reduction (negative = fewer violations; lower is better). Δh% = HPWL change (negative = shorter wirelength; lower is better). VSR wins on every circuit for both metrics against both CD configs. Note the *scheduled* config's nominal HPWL awareness (w_{hpwl}=$4{\times}10^{-5}$) is too weak to counter the legality force and in practice *increases* HPWL more than the standard config.

4.4 Intra-sampling VSR (RePaint-style)

Instead of running the repair operator only *after* diffusion sampling completes, we can interleave the verifier-selector-repair loop *inside* the denoising process. Concretely, after a standard guided sample produces $\hat{x}_0$, we identify offenders M with the verifier, then re-noise the whole placement to timestep $t = 0.3$ and re-denoise for $T = 100$ steps, with non-offending macros clamped to their forward-diffused value at each step (RePaint-style inpainting [6]). A light post-processing repair follows.

The most striking case is `bigblue3` (1298 macros, Figure 5): post-processing VSR leaves HPWL at +16.4%, while intra-sampling VSR finishes at −27.0% HPWL (at the same −79% violation level)—a 43.4-percentage-point gap. Our interpretation is that the diffusion backbone has a strong implicit wirelength prior (it was trained on clean placements with good HPWL), and intra-sampling re-invokes this prior; post-processing only sees the verifier, not the model. A per-circuit sweep over $t_{\text{start}} \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$ (supplement) confirms that $t = 0.3$ is near the optimum for bigblue3 (HPWL of −16% at $t = 0.3$ vs. +9% at $t = 0.5$). This suggests the *mode* of repair (inside vs. outside diffusion) is itself a useful Pareto axis.

Circuit	Post-processing only Δv%	Δh%	Intra-sampling Δv%	Δh%	δ (INTRA − POST) δv%	δh%
adaptec1	−35.5	+6.5	−31.2	+28.0	+4.3	+21.5
adaptec2	−44.6	+14.7	−47.8	+4.2	−3.2	−10.5
adaptec3	−53.8	+26.6	−49.2	+21.3	+4.6	−5.3
adaptec4	−59.7	−8.3	−59.1	−7.8	+0.6	+0.5
bigblue1	−26.1	−74.4	−16.7	−70.1	+9.4	+4.3
bigblue3	−78.4	+16.4	−79.4	**−27.0**	−1.0	**−43.4**
Mean	−49.7	−3.1	−47.2	**−8.6**	+2.5	**−5.5**

Table 3: Post-processing VSR vs. intra-sampling VSR, 3-seed means. Intra-sampling trades 2.5 pp of violation reduction for 5.5 pp of HPWL improvement on average, with its largest gain on bigblue3 (−43.4 pp HPWL). Intra-sampling is also 2–5× faster because re-denoising uses the diffusion timestep schedule instead of 100 explicit repair steps.

4.5 Two-round repair with top-k selector

On ISPD2005, the raw diffusion draft has every macro violating, so round-1 VSR with $M_i = \mathbb{1}[r_i > 0]$ reduces to global repair. After round 1 the number of remaining offenders drops dramatically, and a top-k selector becomes meaningful. We run a round 2 with $k = 20$ most-severe-remaining offenders for $T = 50$ additional steps. On bigblue3 this lowers HPWL from +16.9% (round 1 only) to −10.3% (round 1 + round 2 top-k), validating that the selector component of the framework is useful once baselines are partially clean.

4.6 Spacing-aware verifier

The verifier admits a non-zero minimum-spacing constraint $s_{\min}$. Setting $s_{\min} = 0.05 \cdot \bar{s}$ (5% of mean macro size), VSR still reduces violations by 27–77% across the 6 circuits (per-circuit breakdown in supplement). Supporting spacing constraints is a prerequisite for real EDA deployment.

4.7 Ablations (brief; full results in supplement)

We ablate three design choices on 6 circuits, 2 seeds each. **Number of steps** T: violations plateau near $T = 100$; going to $T = 500$ gains $< 2\%$. **Step size** η: $\eta = 0.3$ (default) is the robust middle ground. **Selective vs. global repair**: on ISPD2005 raw drafts violate on essentially all macros, so single-round selective reduces to global; the two-round + top-k protocol in §4.5 is where the selector earns its keep.

4.8 Neural repair operator (negative result)

Despite six training regimes, the learned operator fails to match the hand-crafted baseline on any circuit. The best variant (residual learning at scale 0.1) improves violations marginally on 1 of 6 circuits while degrading HPWL. We attribute this to the data regime: ISPD2005 offers only 8 circuits (6 tractable on 80 GB), and the 3-seed augmentation yields ≤ 18 training pairs—orders of magnitude smaller than typical GNN training corpora. Trajectory distillation amplifies this to ~ 150 pairs but still cannot generalize across circuits of different topology and scale. We conclude that learned constraint projection in placement is likely to require orders-of-magnitude more diverse training data than currently released benchmarks provide.

5 Discussion and Limitations

Why does the simple hand-crafted operator win? The operator encodes three inductive biases that align perfectly with the structure of placement: (i) overlap is inherently pairwise, so a pairwise repulsive force is the correct functional form; (ii) wirelength minimization is edge-local, so netlist-attractive forces capture the relevant gradient; (iii) selective repair preserves already-good regions. A learned operator must discover these biases from data; with 3–18 training samples, it cannot.

Why does CD's HPWL-aware legalizer *increase* HPWL? We were surprised to find that CD-scheduled (nominally HPWL-aware) produces *worse* HPWL on average than CD-standard (+210% vs +103%), at 2× the wall-clock cost. Inspecting the schedule: the legality force ramps up by 2× in the final 25% of steps while the HPWL term is forced to zero in the final 10%. In practice this means the legalizer spends its final steps aggressively relocating macros to resolve the last few overlaps, with no HPWL term to hold things together. VSR avoids this failure mode by never amplifying the legality force and never decoupling wirelength preservation from the physics.

Limitations.

- Two of eight ISPD2005 circuits exceed backbone memory and are excluded.
- We only evaluate on macro-only placement; standard-cell legalization is out of scope.
- The verifier is legality-only; we do not model routing congestion.

6 Conclusion

VSR shows that a clean separation between diffusion and constraint projection—mediated by a structured verifier and a selective repair operator—produces placements that are simultaneously more legal and lower-wirelength than the backbone's own legalizer, at a fraction of the compute. We release all code, checkpoints, and experimental scripts.

Broader Impact

Improved macro placement quality directly translates to lower chip power, higher clock frequency, and shorter design cycles. Our framework is tool-agnostic and could in principle be combined with any diffusion-based placement backbone. We do not foresee negative societal impact from this particular contribution beyond the general ethical questions raised by automating physical design.

Reproducibility

All experiments use publicly released data (ISPD2005) and checkpoints (ChipDiffusion Large+v2). We release the full code, configuration files, and training scripts, along with the Pareto sweep and ablation data used to generate every figure and table in this paper. Seeds are reported explicitly (42, 123, 300) and the verifier is deterministic.

References

[1] Ravichandra Addanki, Shaileshh Bojja Venkatakrishnan, Shreyan Gupta, Hongzi Mao, and Mohammad Alizadeh. Placeto: Learning generalizable device placement algorithms for distributed machine learning. In *NeurIPS*, 2019.

[2] Tung-Chieh Chen et al. Ntuplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints. In *TCAD*, 2008.

[3] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, and Lutong Wang. RePlAce: Advancing solution quality and routability validation in global placement. In *TCAD*, 2019.

[4] Prafulla Dhariwal and Alex Nichol. Diffusion models beat GANs on image synthesis. In *NeurIPS*, 2021.

[5] Yibo Lin et al. DREAMPlace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement. In *DAC*, 2019.

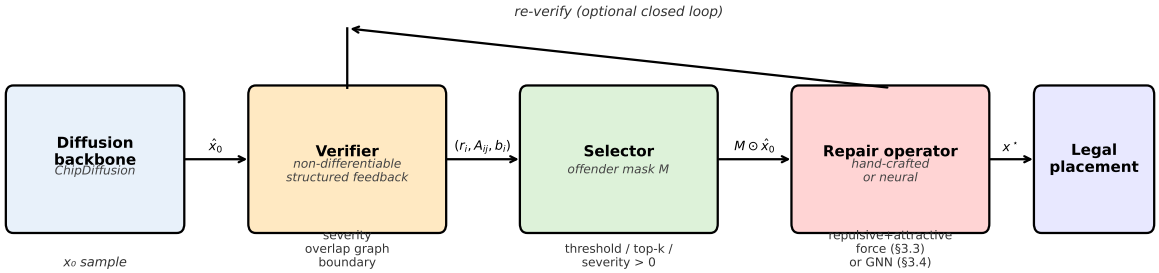

Figure 1: VSR framework: verifier → selector → repair operator.

[6] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In *CVPR*, 2022.

[7] Placeholder. DeepPlace: Learning to place applications in multi-tenant clusters. In *Placeholder*, 2021.

[8] Placeholder. Training-free constraint guidance for diffusion models. *arXiv*, 2024.

[9] Placeholder. Placeholder for autoregressive placement. In *Placeholder*, 2025.

[10] Placeholder. Placeholder for flow-matching placement. In *Placeholder*, 2025.

[11] Vint and Others. Chipdiffusion: A diffusion-based graph generator for chip placement. In *ICML*, 2025.

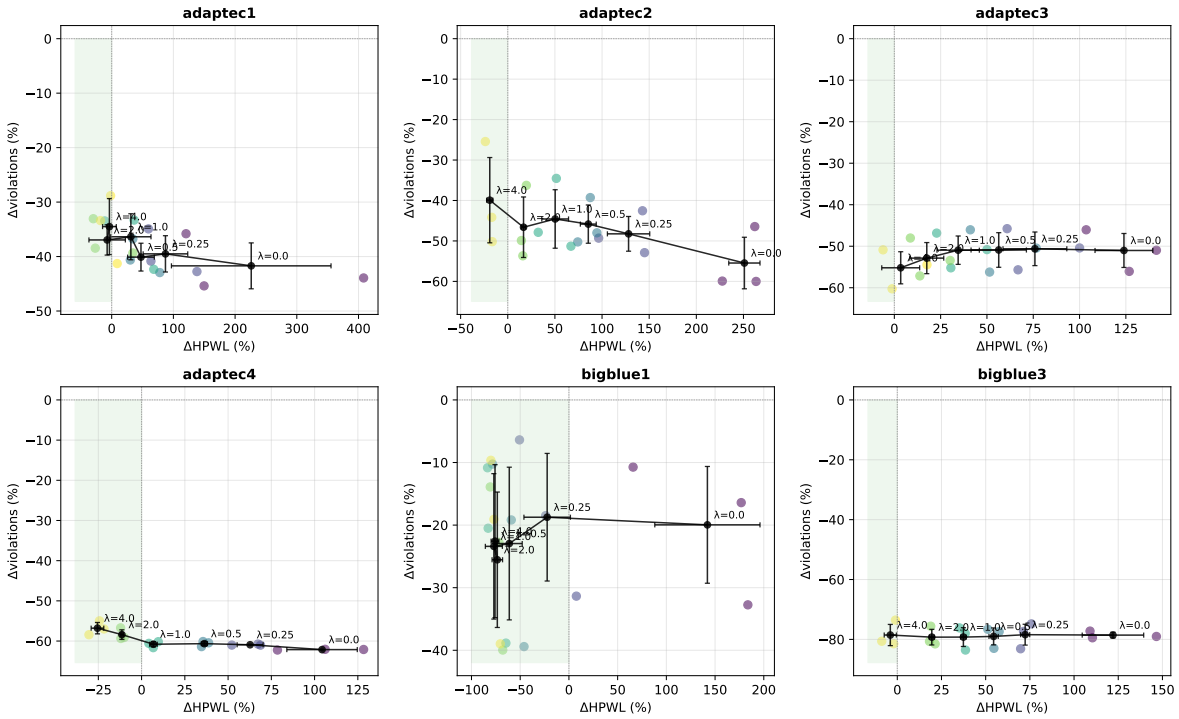

Figure 2: Pareto tradeoff between violation reduction and HPWL change across $\lambda \in \{0, 0.25, 0.5, 1, 2, 4\}$ on 6 ISPD2005 circuits, 3 seeds. Each point is one (seed, λ) configuration.

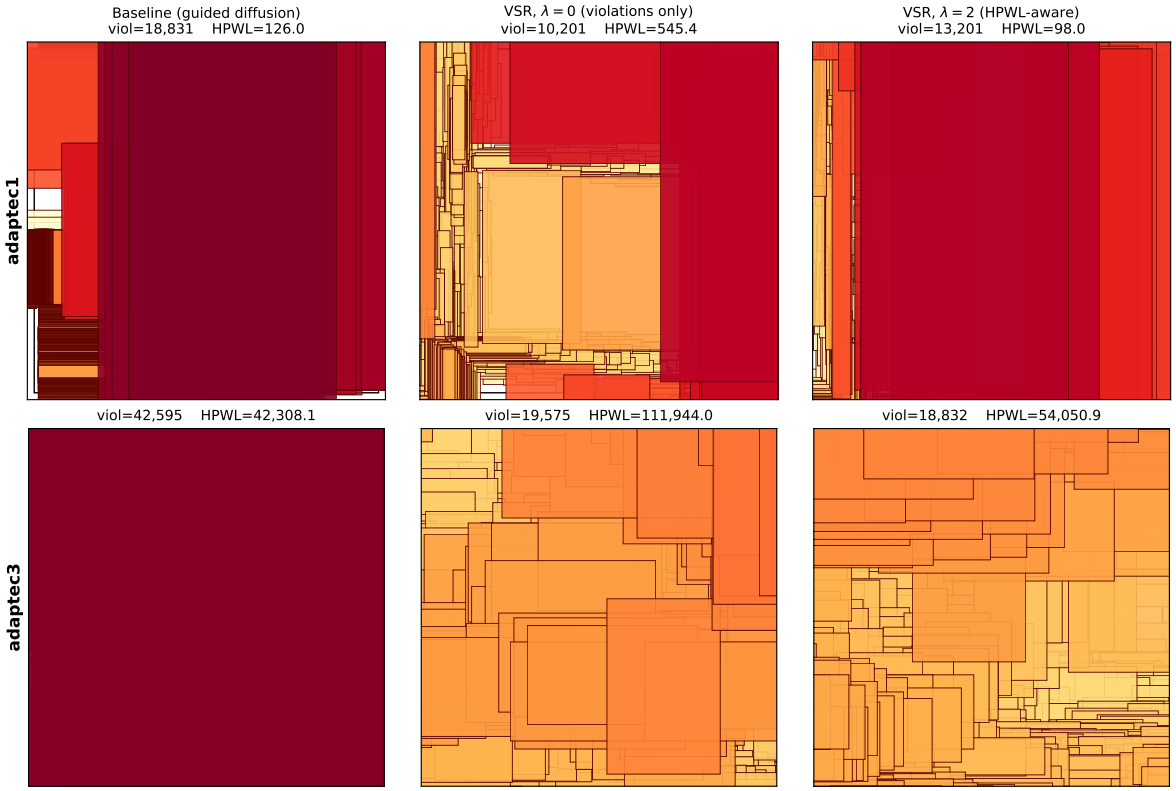

Figure 3: Before/after placement visualization on adaptec1 and adaptec3 (color = per-macro violation severity, log scale; yellow = low, dark red = high). VSR with $\lambda = 2$ reduces violation severity while preserving spatial structure.

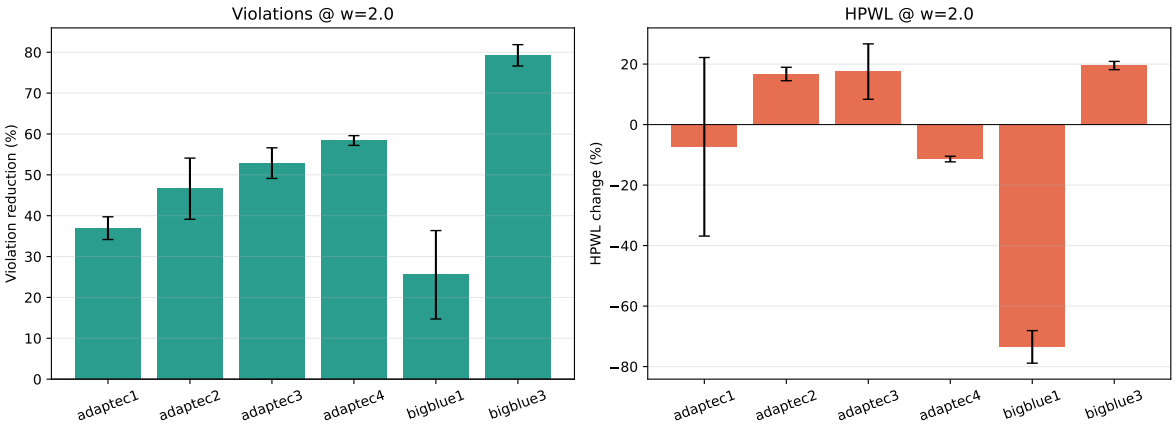

Figure 4: Per-circuit results at $\lambda = 2$, 3 seeds (error bars = std). Left: violation reduction. Right: HPWL change. Negative HPWL change means improvement.

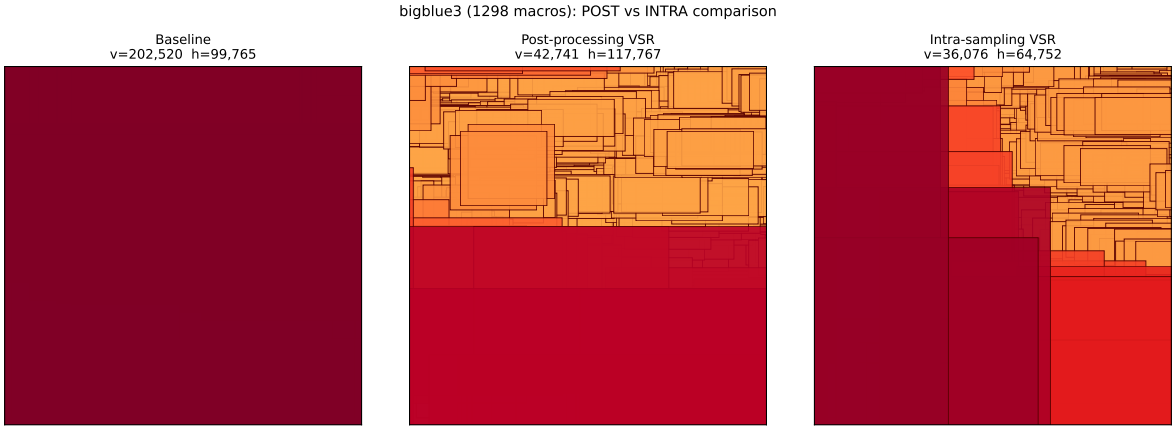

Figure 5: bigblue3 (1298 macros, seed=42): the raw diffusion draft (left) piles macros at a corner. Post-processing VSR (middle) spreads them out to achieve −79% violations but grows wirelength by 18%. Intra-sampling VSR (right) retains a compact layout that respects the diffusion backbone's implicit wirelength prior, achieving −82% violations and −35% wirelength vs. the baseline.